MANCHESTER 1824
The University of Manchester

COMP15111 Fundamentals of Computer Architecture

If and Switch Statements

Luca Peres
luca.peres-2@manchester.ac.uk

Overview and Learning Outcomes

- If statements and switch statements
  - Implemented as: if … else if …
  - Using an address table

- Signed and Unsigned integers

Conditional statements

If-then-else constructs control the execution flow and test conditions

```
if condition
      #do something
else
      #do something else
```

What if there are many possible conditions?

Conditional statements

If-then-else constructs control the execution flow and test conditions

```
if condition_1
      #do something
else if condition_2
      #do something else
else if condition_3
      #do something else
...
else if condition_n
      #do something else
else
      #any other case
```

What if there are many possible conditions?

Conditional statements

If-then-else constructs control the execution flow and test conditions

```
if condition_1
      #do something
else if condition_2
      #do something else
else if condition_3
      #do something else
...
else if condition_n
      #do something else
else
      #any other case
```

```
if number == 0:
   print(‘0’)
elif number == 2:
   print(‘2’)
elif number == 4:
   print(‘4’)
else:
   print(‘?’)
```

First Approach: Nested ifs

- Using compare and branch instructions

```
        lw    t0, number

cmp0    bnez  t0, cmp2
        li    a0, '0'
        li    a7, 0
        ecall
        j end
cmp2    li    t1, 2
        bne   t0, t1, cmp4
        li    a0, '2'
        li    a7, 0
        ecall
        j end
```

```
cmp4    li    t1, 4
        bne   t0, t1, none
        li    a0, '4'
        li    a7, 0
        ecall
        j end

none    li    a0, '?'
        li    a7, 0
        ecall

end     ...
        ...
number  DEFW 0
```

```
if number == 0:
    print('0')
elif number == 2:
    print('2')
elif number == 4:
    print('4')
else:
    print('?')
```

Observations

+ We already know how to do if..else...

- Speed: a large nested if statement can have many cases so:
    - many tests needed to get to cases near the bottom
    - slow
    - lots of code

- Some improvement possible:
    - Note that ASCII codes are numeric
    - code including ecall is repeated

Simplified if… else…

```
ascii_0 equ 48                  ; ascii code for ‘0’
        lw   a0, number         ; loads the numerical value
        addi a0, a0, ascii_0    ; convert to ascii code

cmp0    li   a1, 48             ; compare to ascii ‘0’
        beq  a0, a1, print
cmp2    li   a1, 50             ; compare to ascii ‘2’
        beq  a0, a1, print
cmp4    li   a1, 52             ; compare to ascii ‘4’
        beq  a0, a1, print
        li   a0, 63             ; none of them, set to ascii ‘?’
print   li  a7, 0
        ecall
        ...
number defw 0
```

If-else Method

Advantages and disadvantages of using “If-else” method

Advantages	Disadvantages
1. Simple and direct implementation 2. Can handle non-numeric and sparse input variables without any memory overhead	1. Many comparisons and branch instructions are required to get to the last of the if-else statements 2. Takes more time to execute if there are large number of nested if-else statements

Observations

- Code has been simplified somewhat but:
  - We still may have to make many tests to get to cases near the bottom
  - There are more even efficient ways

Conditional statements

Switch statements can do this in a better way!

```
switch var
     condition_1: #do something
     condition_2: #do something else
     condition_3: #do something else
     ...
     condition_n: #do something else
     default:     #any other case
```

Python Switch

Python does not have native switch statements
We can implement equivalent by using dictionary mappings

```
if number == 0:
   print(‘0’)
elif number == 2:
   print(‘2’)
elif number == 4:
   print(‘4’)
else:
   print(‘?’)
```

```
def switch(number):
    mapping = {
        0 : ‘0’,
        2 : ‘2’,
        4 : ‘4’
    }
    print (mapping.get(number,‘?’))
```

Tables Recap

Tables are a convenient way of storing data

Number	RGB in hex	Name
0	#00 00 00	black
1	#80 00 00	maroon
2	#00 80 00	green
3	#80 80 00	olive
4	#00 00 80	navy
5	#80 00 80	purple
6	#00 80 80	teal
7	#c0 c0 c0	silver
8	#80 80 80	grey

```
table

defw 0x000000 ; black

defw 0x800000 ; maroon

defw 0x008000 ; green

defw 0x008080 ; teal

defw 0xc0c0c0 ; silver

defw 0x808080 ; grey
```

The colour "olive" can be loaded with

```
la  t0, table
lw  t1, 12[t0]
```

Tables Recap

What if we use them for addresses?

Number	Code address
0	`condition_1`
1	`condition_2`
2	`condition_3`
…	`...`
n	`condition_n`
n+1	`default`

Labels are addresses.
They can point to variables or code too!

`bnez t0, condition_3`
Branches to an address!

Address Table

Can we implement a Switch with a table?

- Yes!
- Use a table (list) holding the addresses of the pieces of code
    - The table has one word for each possible value of the variable
    - Each table entry is the address of the code to be executed for that value
    - Use the value of the variable to find the right address in the table
    - …ie the variable is treated as an index to the table.

Table pseudo-code

```
        if (number<0 || number>4) j default;
        j number'th word in the table;

table: defw case0   ; word 0 of table at table+0
       defw default ; word 1 of table at table+4
       defw case2   ; word 2 of table at table+8
       defw default ; word 3 of table at table+12
       defw case4   ; word 4 of table at table+16

case0: print("0")
       j end;
case2: print("2")
       j end;
case4: print("4")
       j end;
default: print("?")
end
```

RISC-V Code for each case

```
case0   li  a0, '0'
        li  a7,  0
        ecall
        j   end
case2   li  a0, '2'
        li  a7,  0
        ecall
        j   end
case4   li  a0, '4'
        li  a7,  0
        ecall
        j   end
default li  a0, '?'
        ecall
end     . . .
```

RISC-V Code for the Switch

```
        lw   a0, number
        bltz a0, default ; trap values < 0
        lw   a1, 4       ; trap values > 4
        bgt  a0, a1, default

        la   a1, table
        slli a0, a0, 2
        add  a1, a1, a0
        jalr zero, 0[a1] ; table + (number*4)

table   defw case0       ; address = table + 0
        defw default     ; address = table + 4
        defw case2       ; address = table + 8
        defw default     ; address = table + 12
        defw case4       ; address = table + 16
        defw default     ; address = table + 20
```

Example

e.g. if number = 2

Register	value
a0	number
a1	table
...	
...	

Address	value	
00000000	...	main
00000004	...	main
	slli a0, a0, 2	main
	add a1, a1, a0	main
	jalr zero, 0[a1]	main
end	...	
case0	li a0, ‘0’	cases
case0+4	li a7, 0	cases
case0+8	ecall	cases
case0+12	j end	cases
case2	li a0, ‘2’	cases
case2+4	li a7, 0	cases
case2+8	ecall	cases
case2+12	j end	cases
...	...	cases
default	li a0, ‘?’	cases
...	...	cases
table+0	case0	table
table+4	default	table
table+8	case2	table
...	...	table

Observations

+ Table size doesn't affect speed

- Table lookup is a (small) overhead
- Table lookup is more complex if:
  - Cases are sparsely distributed
  - Switch variable is non-numeric

This is a simple example of using a table of addresses

We do something similar:

- to implement objects
- to handle peripherals (later)
- to implement libraries (classes) of useful methods etc.

Address Table Method

Advantages and disadvantages of using “Address table” method

Advantages	Disadvantages
1. Table based implementation takes less time to execute as compared to the direct “if-else” method 2. The numeric variable can be used to directly index the table 3. Require fewer branch instructions	1. Table lookup is more complex when the input variable is non-numeric 2. Memory footprint is very high when if-else statements are sparsely placed

Types & Representations - recap

- Every value is represented as a bit-pattern
- e.g. bit-patterns for 0 and false may be the same
- Python compiler checks types for us
- RISC-V (assembler) doesn't
- e.g. can add characters (remember last example class?)

Signed and Unsigned integers

Assume 8-bit numbers (instead of 32-bit) so 256 (2^8) different values

Unsigned values 0 to 255 (base-2 notation)
Signed values −128 to +127 (2's-complement)

0x00 to 0x7F (0000 0000 to 0111 1111)
– same representation for 0 to 127:

MSB indicates the sign for signed numbers

0x80 to 0xFF (1000 0000 to 1111 1111) represent:
– unsigned values 128 to 255
– signed values −128 to −1

Bit-patterns and numbers

0111 1111

1000 0000

8-bit-patterns:

0x00 0x01 ... 0x7E 0x7F 0x80 0x81 ... 0xFE 0xFF

signed values:

0 1 ... 126 127 −128 −127 ... −2 −1

unsigned values:

0 1 ... 126 127 128 129 ... 254 255

32-bit-patterns: 0x00000000 ... 0x7FFFFFFF 0x80000000 ... 0xFFFFFFFF

signed values:

0 ... $2^{31} - 1$ -2^{31} ... −1

unsigned values:

0 ... $2^{31} - 1$ 2^{31} ... $2^{32} - 1$

Table Optimisation: initial tests

check (signed) value: $0 \leq n \leq 4$
i.e. 0 to 4 valid, every other value invalid

signed: fail if n >4 (i.e. 0x00000005 to 0x7FFFFFFF) or n < 0 (i.e. 0x80000000 to 0xFFFFFFFF)

unsigned: fail if n >4 (i.e. 0x00000005 to 0xFFFFFFFF)

Same for blt, bge

```
bltz a0, default     ; signed test “Less Than”
lw   a1, 4
bgt  a0, a1, default ; signed test “Greater Than”
```

→

```
lw   a1, 4
bgtu a0, a1, default ; unsigned test “Higher than”
```

Table Optimisation: initial tests

```
        lw   a0, number
; The numbers are signed, but if we use an unsigned
; test negative numbers appear to be >128
        lw   a1, 4
        bgtu a0, a1, default ; number < 0 OR number > 4
; Saved a couple of instructions but the way that
; the code functions is obscured. Is it worth it?

        la   a1, table
        slli a0, a0, 2
        add  a1, a1, a0
        jalr zero, 0[a1]

table defw case0            ; address = table + 0
      defw default          ; address = table + 4
      defw case2            ; address = table + 8
      defw default          ; address = table + 12
      defw case4            ; address = table + 16
      defw default          ; address = table + 20
```

Question

What is the RISC-V code for the start of this switch statement:)?
(assume that t1 already holds the base address of the table and t0 holds the number)

```
def switch(arg):
    mapping = {
        -1 : "-1",
         0 : "0",
         1 : "1"
    }
    print mapping.get(arg,"?")
```

One solution

```
      li  a1, -1; trap values < -1
      blt a0, a1, default
      li  a1,  1; trap values > 1
      bgt a0, a1, default

      la  a1, table
      slli a0, a0, 2
      add  a1, a1, a0
      jalr zero, 0[a1]

      defw case_m1    ; address = table - 4
table defw case0      ; address = table
      defw case_p1    ; address = table + 4
      defw default    ; address = table + 8
```

Is there another way?

Try adding 1 to `number`:

```
def switch(arg+1):
    mapping = {
         0 : "-1",
         1 : "0",
         2 : "1"
    }
    print mapping.get(arg,"?")
```

Summary

- Switch statements
  - If … else if …
  - Tables
- Signed and Unsigned Integers

MANCHESTER 1824
The University of Manchester

COMP15111 Fundamentals of Computer Architecture

Booleans

Luca Peres
luca.peres-2@manchester.ac.uk

Overview and Learning Outcomes

- Booleans and Bitwise operations
- Booleans from Comparisons
- Lazy evaluation
- Bitwise operations for setting, clearing and testing individual bits

Symbols

Follow Python conventions for symbols and precedence.

A | B & C ^ D means
A | ((B & C) ^ D)

Operator	Description
`lambda`	Lambda expression
`if - else`	Conditional expression
`or`	Boolean OR
`and`	Boolean AND
`not x`	Boolean NOT
`in, not in, is, is not, <, <=, >, >=, !=, ==`	Comparisons, including membership tests and identity tests
`|`	Bitwise OR
`^`	Bitwise XOR
`&`	Bitwise AND
`<<, >>`	Shifts
`+, -`	Addition and subtraction
`*, @, /, //, %`	Multiplication, matrix multiplication, division, floor division, remainder [5]
`+x, -x, ~x`	Positive, negative, bitwise NOT
`**`	Exponentiation [6]
`await x`	Await expression
`x[index], x[index:index], x(arguments...), x.attribute`	Subscription, slicing, call, attribute reference
`(expressions...), [expressions...], {key: value...}, {expressions...}`	Binding or tuple display, list display, dictionary display, set display

Lowest

highest

https://docs.python.org/3/reference/expressions.html#summary

Booleans - examples

```
if w < x:  . . . # w and x are integers

b1, b2, b3
b1 = w < x
b2 = w < x and w > y
```

precedence: >,< before and,or

```
b2 = (w < x) and (w > y)
b3 = w < x or w > y
b3 = (w < x) or (w > y)

if w < x and w > y:  . . .  Boolean AND (and)
if w < x or w > y:  . . .   Boolean OR  (or)


w = (w | !x) & (y ^ z)... Bitwise OR (|)
                          Bitwise AND (&)
```

Representing Boolean Variables

Tradeoff: Space vs. Time

Boolean (true/false) = 1 bit
→ could pack 8 booleans into a byte

Extracting 1 bit from a byte usually takes extra instructions Normally don't use

lots of booleans, but do want fast code,

→ use a byte for each Boolean variable

Usual representation: **false** is 0 (0x00), **true** is 1 (0x01) (although you can't see this directly in Python)

For interest:
In the C language there is no 'boolean' type and this is *explicit*! Anything *non-zero* is interpreted as true.

Boolean Expressions

e.g.

```
if x == 0:
  print("x is zero")
```

Doesn't need explicit boolean variable

- just bne (or bnez…)

```
lw  t0, x
bne t0, zero, false
...
```

```
bnez t0, false
```

RISC-V "Bitwise" Logical Operations

Logical or Boolean operations are implemented bitwise in RISC-V:

```
; e.g.          t1=0b11001100
;               t2=0b00001111
and t0,t1,t2;   t0=0b00001100
or  t0,t1,t2;   t0=0b11001111 ;
xor t0,t1,t2;   t0=0b11000011 ; (Exclusive OR, XOR, )
not t0,t2;      t0=0b11110000 ; NOT (!) invert bits
```

```
not t0, t2 = xori t0, t2, -1
```

t2=0b00001111
-1=0b11111111
xori =0b11110000

A	B	AND=A&B	OR=A|B	XOR=A^B	!A
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

Simple Booleans: **take care**, !00000001 = 11111110 ≠ 0

Examples

```
b4 = (b1 | !b2) & b3 ^ b4; b=0 ⇒ false, b=1 ⇒ true
...
if (b4) ...

lb    t0, b1
lb    t1, b2
xori  t1, t1, -1 ;  b2 ^ -1 = !b2 (equivalent to not)
or    t0, t0, t1 ;  b1 | !b2
lb    t1, b3
and   t0, t0, t1 ; (b1 | !b2) & b3
lb    t1, b4
xor   t0, t0, t1 ; ((b1 | !b2) & b3) ^ b4
sb    t0, b4, t2

...
lb    t0, b4
beqz  false
```

Translate in RISC-V

```
b4 = b1 & b2 | !b3;

b4 = (b1 & b2) | (!b3);
```

```
lb   t0, b1
lb   t1, b2
and  t0, t0, t1 ;  b1 & b2
lb   t1, b3
xor  t1, t1, -1 ; b3 ^ 1 = !b3
or   t0, t0, t1 ; b1 & b2 | !b3
sb   t0, b4, t2
```

Operator	Description
`lambda`	Lambda expression
`if - else`	Conditional expression
`or`	Boolean OR
`and`	Boolean AND
`not x`	Boolean NOT
`in`, `not in`, `is`, `is not`, `<`, `<=`, `>`, `>=`, `!=`, `==`	Comparisons, including membership tests and identity tests
`|`	Bitwise OR
`^`	Bitwise XOR
`&`	Bitwise AND
`<<`, `>>`	Shifts
`+`, `-`	Addition and subtraction
`*`, `@`, `/`, `//`, `%`	Multiplication, matrix multiplication, division, floor division, remainder [5]
`+x`, `-x`, `~x`	Positive, negative, bitwise NOT
`**`	Exponentiation [6]
`await x`	Await expression
`x[index]`, `x[index:index]`, `x(arguments...)`, `x.attribute`	Subscription, slicing, call, attribute reference
`(expressions...)`, `[expressions...]`, `{key: value...}`, `{expressions...}`	Binding or tuple display, list display, dictionary display, set display

Booleans from Comparisons

```
b = (x<y and y<z) or x>z ;  (x, y, z ints, b boolean)
```

```
lw    t0, x
lw    t1, y
lw    t2, z
slt   t3, t0, t1  ; x<y? t3=1 if true, else t3=0
slt   t4, t1, t2  ; y<z? t4=1 if true, else t4=0
and   t3, t3, t4  ;  t3 ⇐(x<y and y<z)
slt   t4, t2, t0  ; x>z? t4=1 if true, else t4=0
or    t3, t3, t4  ; t3 ⇐(x<y and y<z) or x>z
sb    t3, b, t5
```

Set instructions

```
slt     Rd, r1, r2   ; Rd = 1 if r1 < r2, else Rd = 0
slti    Rd, r1, imm  ; Rd = 1 if r1 < imm, else Rd = 0
sltu    Rd, r1, r2   ; Rd = 1 if r1 < r2, else Rd = 0
sltiu   Rd, r1, imm  ; Rd = 1 if r1 < imm, else Rd = 0
```

```
seqz    Rd, rs   ; Rd = 1 if rs = 0, else Rd = 0
snez    Rd, rs   ; Rd = 1 if rs ≠ 0, else Rd = 0
sltz    Rd, rs   ; Rd = 1 if rs < 0, else Rd = 0
sgtz    Rd, rs   ; Rd = 1 if rs > 0, else Rd = 0
```

Lazy Evaluation

```
b = (x<y and y<z) or  (x>z)
```

Rewrite to test x<y first:

```
if (x<y): b = (true and y<z) or  (x>z)  # x<y true
else:     b = (false and y<z)or  (x>z)  # x<y false
```

```
if (x<y): b = (y<z) or  (x>z)
else:     b = x>z
```

A	B	AND	OR
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

Lazy Evaluation

```
if (x<y): b = (y<z) or  (x>z) else:
          b = x>z
```

Rewrite to test y<z next:

```
if (x<y) :
  if (y<z):     b = true or (x>z)
  else:          b = false or  (x>z)
else:            b = x>z
```

```
if (x<y):
  if (y<z): b = true
  else:          b = x>z
else:            b = x>z
```

if both x<y and y<z then b=true, but if either is false then b=x>z

Try converting this to RISC-V

A	B	AND	OR
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

RISC-V “Bitwise” Logical Operations

Sometimes (e.g. in I/O or control applications) we do operate logically on individual bits. Looking at 8 bits only:

```
; e.g.            t1=0b11001100
;                 t2=0b00001111
and t0,t1,t2 ;    t0=0b00001100
or  t0,t1,t2 ;    t0=0b11001111
xor t0,t1,t2 ;    t0=0b11000011
not  t0,t2   ;    t0=0b11110000
```

A	B	AND=A&B	OR=A|B	EOR=A^B	BIC=A&!B	!A
0	0	0	0	0	0	1
0	1	0	1	1	0	1
1	0	0	1	1	1	0
1	1	1	1	0	0	0

Instructions for individual bits

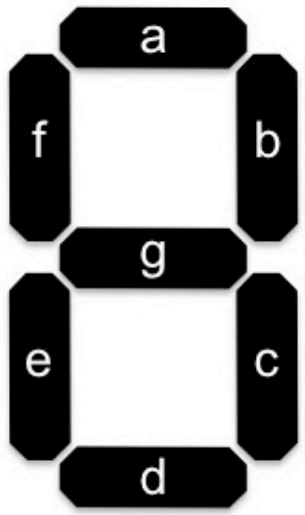
- We need to be able to set (to 1), clear (to 0) and test the values of individual bits in a byte or word:

```
; e.g.            t0 = 0b11001100
;                 Op2= 0b00001111
ori  t1,t0,Op2 ; t1 = 0b11001111
andi t1,t0,Op2 ; t1 = 0b00001100
xori t1,t0,Op2 ; t1 = 0b11000011
```

- ori Rd, Rn, Op2 ; Set those bits that are set in Op2:
  Rn is copied to Rd, any bit in Op2 that is '1' is set to '1' in Rd
- andi Rd, Rn, Op2 ; Clear all bits except those set in Op2:
  Rn is copied to Rd, any bit in Op2 that is '0' is cleared to '0' in Rd
- xori Rd,Rn,Op2 ; Inverts all the bits set in Op2:
  Rn is copied to Rd, any bit in Op2 that is '1' is inverted in Rd

7-segment decoder

- In some applications it is useful for individual bits to have different functions:
- e.g. when connecting a processor to hardware:
    - displays
    - instrumentation
    - control…
- Take as an example a 7-segment display:



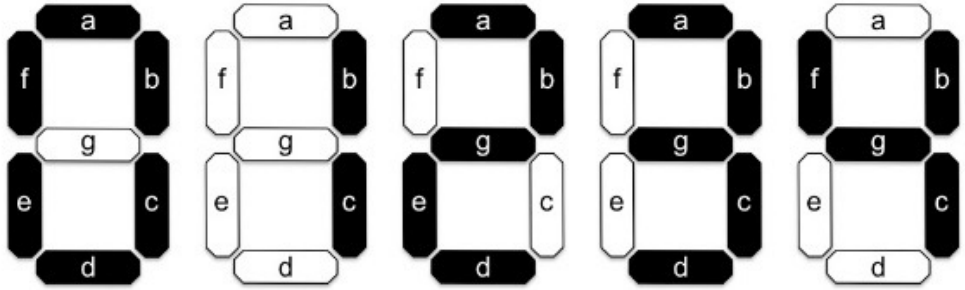
- Each segment is separately controlled.
- One bit per segment
- One byte per digit
- '1' = lit, '0' = not lit
- 0b01111111 displays 8
- 0b00000110 displays 1
- etc (PTO)

Bit number	Segment
0	a
1	b
2	c
3	d
4	e
5	f
6	g

Other uses of bitwise operations

- Assuming that we can illuminate the display by storing a byte at a given address ‘display’.
- display the integer in the LSB of a0:

```
la   a1, segments
add  a1, a1, a0
lb   a2, 0[a1]
sb   a2, display, t0
```



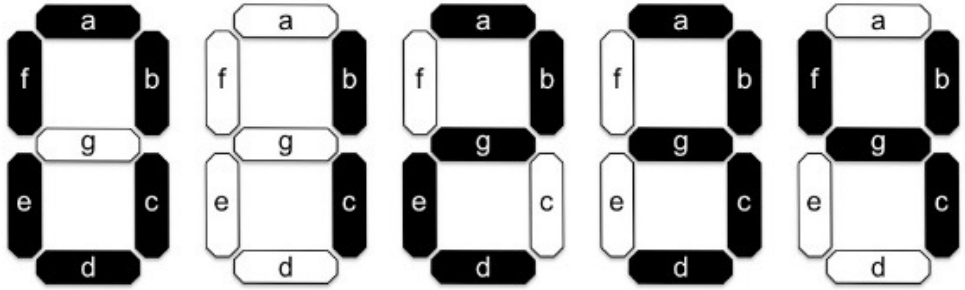
segments: 00111111 ; segment pattern to display 0
00000110 ; segment pattern to display 1
01011011 ; segment pattern to display 2
etc

display ; putting a byte here illuminates the display

What's wrong?

- Can you see anything wrong with this?
- What should be done to fix it?

```
la   a1, segments
add  a1, a1, a0
lb   a2, 0[a1]
sb   a2, display, t0
```



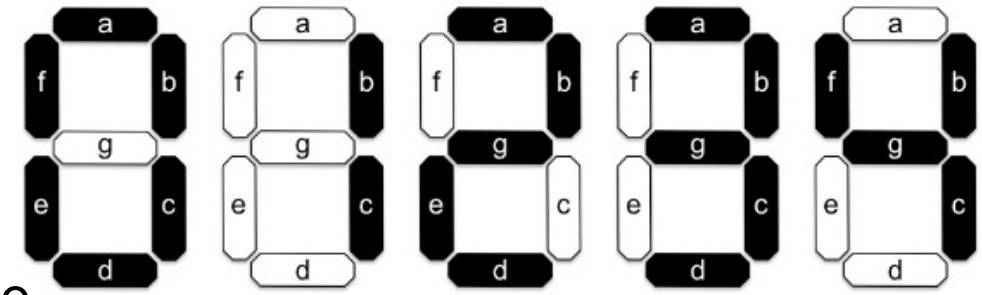
segments: 00111111 ; segment pattern to display 0
00000110 ; segment pattern to display 1
01011011 ; segment pattern to display 2
etc

display ; putting a byte here illuminates the display

Better...

- Can only display one digit (4 bits)
- Clear all bits except the lowest 4 before displaying:

```
la    a1, segments
andi  a0, a0, 0b1111
add   a1, a1, a0
lb    a2, 0[a1]
sb    a2, display, t0
```



segments: 00111111 ; segment pattern to display 0
00000110 ; segment pattern to display 1
01011011 ; segment pattern to display 2
etc

display ; putting a byte here illuminates the display

Summary

- Python: Booleans, operators
- RISC-V bitwise and Booleans operations
- RISC-V instructions
  - And, or, xor, not
- Lazy evaluation
- Bitwise operations